Η ΠΑΡΟΥΣΙΑ ΚΑΙ Η ΑΠΟΥΣΙΑ ΤΗΣ ΡΟΥΤΙΝΑΣ Main

Στις C like γλώσσες, συνηθίζουμε να ορίζουμε μια συνάρτηση με το όνομα **main**. Σε πολλές από αυτές τις γλώσσες προγραμματισμού, η main είναι υποχρεωτική.

Η main <u>είναι προαιρετική</u> στην γλώσσα Python.

Για να εισάγουμε συναρτήσεις ακολουθούμε την λογική

```
def roytina_1(my_1):
        .
        .
        .
        return my_result1

def roytina_2(my_2):
        .
        .
        .
        return my_result2

my_s = 12345
my_s = roytina_1(my_s)
my_s = roytina_2(my_s)
print my_s
```

Μ.Ι.Σ. 2025-2026

ΕΜΒΕΛΕΙΑ ΜΕΤΑΒΛΗΤΩΝ (scope)

Η εμβέλεια των μεταβλητών είναι κάτι πολύ σημαντικό και καθορίζει την εμβέλεια κάθε μεταβλητής μέσα σε ένα πρόγραμμα.

Όταν μια μεταβλητή δηλώνεται μέσα σε μια συνάρτηση είναι ορατή μόνο μέσα σε αυτήν τη συνάρτηση. Αυτό σημαίνει πως η μεταβλητή μπορεί είναι διαθέσιμη μέσα στην ίδια συνάρτηση αλλά δεν μπορεί να προσπελαστεί μέσα σε μια άλλη συνάρτηση.

Υπάρχουν δηλαδή **καθολικές** και **τοπικές** μεταβλητές.

Μεταβλητές μέσα σε μια συνάρτηση, δεν έχουν σχέση με άλλες μεταβλητές που έχουν την ίδια ονομασία αλλά βρίσκονται έξω από την πρώτη, τα ονόματα δηλαδή των μεταβλητών μπορούν να είναι κοινά.

```
my_total = 0                                # καθολική μεταβλητή

def sum(my_a1,my_a2):
        my_total = my_a1 + my_a2            # τοπική μεταβλητή
        print "το α έχει →", my_total
        return my_total

sum(1, 2)

print "το β έχει → ", my_total              # θα μας τυπώσει
                                            #  3
                                            #  0
```

Μ.Ι.Σ. 2025-2

ΤΡΟΠΟΠΟΙΗΤΕΣ ΕΜΒΕΛΕΙΑΣ ΜΕΤΑΒΛΗΤΩΝ

Η εμβέλεια των μεταβλητών μπορεί να επηρεαστεί με τους λεγόμενους τροποποιητές εμβέλειας.

Οι δυνατότητες αλλαγής της συμπεριφοράς εμβέλειας βασίζεται στις δυνατότητες διαχωρισμού ενός Python προγράμματος, εν μέσω των δυνατοτήτων της αντικειμενοστρέφειας το οποίο θα αναλύσουμε στην συνέχεια.

Οι μεταβλητές που δηλώνονται στο βασικό κορμό του προγράμματος (έξω από τις συναρτήσεις) είναι καθολικές μεταβλητές.

Εάν πρέπει μέσα σε μια συνάρτηση να αλλάξει η τιμή μιας καθολικής μεταβλητής τότε πρέπει να δηλωθεί μέσα στην συνάρτηση ότι η μεταβλητή αυτή είναι καθολική (σε όλο το πρόγραμμα).

Αυτή η γενίκευση της εμβέλειας της ανωτέρω μεταβλητής γίνεται με τη χρήση της εντολής **global.**

Η Python διαθέτει και έναν άλλο τροποποιητή εμβέλειας που χαρακτηρίζει μια μεταβλητή σε, μη τοπικής εμβέλειας, και εφαρμόζεται με την εντολή **nonlocal** και βρίσκεται (σαν έννοια) ανάμεσα στη λογική των καθολικών και των τοπικών μεταβλητών.

Οι μεταβλητές τύπου **nonlocal** εμφανίζονται μέσα σε συναρτήσεις που βρίσκονται μέσα σε άλλες συναρτήσεις.

Μ.Ι.Σ. 2025-2026

ΟΙ ΤΥΠΟΙ ΔΕΔΟΜΕΝΩΝ ΣΥΜΒΟΛΟΣΕΙΡΑΣ str

Ο τύπος δεδομένων **str** στην Python είναι για μια σειρά αλφαριθμητικών χαρακτήρων (γενικά εκτυπώσιμων συμβόλων) και είναι στην πραγματικότητα ένας πίνακας χαρακτήρων σε σειρά. Ο τύπος δεδομένων **str** θεωρείται σύνθετος τύπος δεδομένων. Δεν υπάρχει τύπος δεδομένων **char** στην Python.

Όταν λέμε σειρά εννοούμε διαδοχικές θέσεις μνήμης που μπορούν να αντιμετωπισθούν σαν ένα σύνολο διατεταγμένων στοιχειωδών δεδομένων.

Η συμβολοσειρά **str** είναι ένας ειδικός τύπος πίνακα με δεδομένα. Για να οριστούν ή για να ξεχωρίσουν από τις λέξεις κλειδιά της Python τα περικλείουμε μέσα σε εισαγωγικά. Στην Python είναι επιτρεπτό να χρησιμοποιήσουμε μονά (**‘**), διπλά (**“**) ή τριπλά εισαγωγικά (**“‘** ή **“““**).

```
my_string_A = "periexei"
```

το ανωτέρω έχει δημιουργήσει ένα πεδίο τύπου **str** σαν έναν πίνακα 8 θέσεων

και για να εκτυπωθεί, θα πρέπει να δοθεί,

η εντολή, **my_string_A[0]** #που θα τυπώσει το p

η εντολή, **my_string_A[5]** #που θα τυπώσει το x

η εντολή, **my_string_A[2]** #που θα τυπώσει το r

γίνεται δηλαδή μια κλασσική προσπέλαση με δείκτες. Η αρίθμηση του δείκτη γίνεται από το μηδέν, ενώ υπάρχει δυνατότητα για χρήση αρνητικού δείκτη, οι οποίοι μετράνε ανάποδα, από το τέλος της συμβολοσειράς προς την αρχή.

η εντολή, **my_string_A[-2]** #που θα τυπώσει επίσης το x

Στην Python έχουμε επίσης τη δυνατότητα να πάρουμε μέρος της συμβολοσειράς με μια ειδική δήλωση στον δείκτη.

για την συμβολοσειρά

```
my_string_A = "periexei"
```

η εντολή, **my_string_A[5:7]** #θα τυπώσει το xe (όχι την θέση 7)

η εντολή, **my_string_A[0:2]** #θα τυπώσει το pe (όχι την θέση 2)

επιπλέον εάν στην αμέσως ανωτέρω δήλωση γράψουμε

my_string_A[:2] #θα τυπώσει επίσης το pe (όχι την θέση 7)

και τέλος εάν γράψουμε

my_string_A[5:] #θα τυπώσει το xei

Στην Python οι συμβολοσειρές είναι αμετάβλητες και το περιεχόμενο τους δεν μπορεί να αλλάξει με μια απλή ανάθεση με δείκτη, η δομή τους ανήκει στις μη μεταβαλλόμενες (immutable) δομές της Python,

δηλαδή η δήλωση,

my_string_A[0:1] = «Φ»

δεν είναι αποδεκτή.

Μ.Ι.Σ. 2025-2026

ΒΑΣΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ΓΙΑ ΣΥΜΒΟΛΟΣΕΙΡΕΣ

Η Python διαθέτει «εργοστασιακές» συναρτήσεις για επεξεργασία συμβολοσειρών, διαφόρων χρήσεων,

- **my_s.isalpha()** επιστρέφει TRUE εάν η μεταβλητή my_s, περιέχει μόνο γράμματα
- **my_s.isdigit()** επιστρέφει TRUE εάν η μεταβλητή my_s, περιέχει μόνο ψηφία
- **my_s.islower()** επιστρέφει TRUE εάν η μεταβλητή my_s, περιέχει μόνο πεζά γράμματα
- **my_s.isupper()** επιστρέφει TRUE εάν η μεταβλητή my_s, περιέχει μόνο κεφαλαία γράμματα
- **my_s.isprintable()** επιστρέφει TRUE εάν η μεταβλητή my_s, περιέχει μόνο εκτυπώσιμους χαρακτήρες
- **my_s.isspace()** επιστρέφει TRUE εάν η μεταβλητή my_s, περιέχει μόνο (κενά, tab, αλλαγές γραμμής)
- **my_s.startswith(my_t)** επιστρέφει TRUE εάν η μεταβλητή my_s, ξεκινά με την my_t
- **my_s.endswith(my_t)** επιστρέφει TRUE εάν η μεταβλητή my_s, τελειώνει με την my_t
- **my_s.find(my_t)** επιστρέφει τον πρώτο δείκτη, στον οποίο ξεκινά η my_t μέσα στην my_s, αλλιώς -1
- **my_s.count("δ")** μετρά, πόσα «δ», υπάρχουν μέσα στην my_s

Μ.Ι.Σ. 2025-2026

- **my_s.lower()** μετατρέπει όλα τα γράμματα σε πεζά
- **my_s.upper()** μετατρέπει όλα τα γράμματα σε κεφαλαία
- **my_s.swapcase()** μετατρέπει τα πεζά σε κεφαλαία και τα κεφαλαία σε πεζά
- **my_s.strip(er)** αφαιρεί τα er που εμφανίζονται στην αρχή ή το τέλος της my_s
- **my_s.lstrip(er)** αφαιρεί τα er που εμφανίζονται στην αρχή της my_s
- **my_s.rtrip(er)** αφαιρεί τα er που εμφανίζονται στο τέλος της my_s
- **lenght(my_s)** μετρά πόσους χαρακτήρες έχει μέσα της η, my_s
- **strt(33) == “33”** μετατρέπει μια τιμή σε συμβολοσειρά (θα βγάλει TRUE)
- **int("33") == 33** μετατρέπει ένα αλφαριθμητικό στον ακέραιο αριθμό που αναπαριστά (θα βγάλει TRUE)

καθώς υπάρχουν πολλές σχετικές συναρτήσεις για τις συμβολοσειρές και τον χειρισμό τους η εντολή **help(str)** θα τυπώσει όλες τις διαθέσιμες επιλογές.

πριν από κάθε μέθοδο/ρουτίνα/συνάρτηση, υπάρχει μια τελεία, η οποία σημαίνει, εκτέλεσε την ρουτίνα επάνω σε αυτό που προηγείται της τελείας.

Μ.Ι.Σ. 2025-2026

ΠΑΡΑΔΕΙΓΜΑΤΑ ΣΥΝΑΡΤΗΣΕΩΝ ΕΠΕΞΕΡΓΑΣΙΑΣ ΣΥΜΒΟΛΟΣΕΙΡΩΝ

1) δημιουργήστε ένα πρόγραμμα με μια συνάρτηση που μετρά το πλήθος των a,e,o που περιέχει μια συμβολοσειρά.

```
def count_them(the_pass_keimeno):
        my_grammata = "aeo"
        my_count = 0
        for my_letter in the_pass_keimeno:
                if my_letter in my_grammata:
                        my_count += 1
        return my_count

my_keimeno=input("πληκτρολογήστε κείμενο…")

print "υπάρχουν…", count_them(my_keimeno), "…a,e,o"
```

Μ.Ι.Σ. 2025-2026

2) δημιουργήστε ένα πρόγραμμα που δέχεται μια συμβολοσειρά και με μια συνάρτηση αρχικά αλλάζει τα κενά με το «+», με μια δεύτερη συνάρτηση επιστρέφει τους 10 πρώτους χαρακτήρες της,

```
def bale_kena(my_sentence):
    my_result = ""
    for my_char in my_sentence :
        if my_char != " " :
            my_result += my_char
        else:
            my_result += "_"
    return my_result


def fere_10(my_sentence):
    if len(my_sentence)<=10:
        my_result = my_sentence
    else:
        my_result = ""

    for my_i in range(10):
        my_result += my_sentence[my_i]

    return my_result
```

Μ.Ι.Σ. 2025-2026

```
my_s = input("String: ")
my_s = bale_kena(my_s)
my_s = fere_10(my_s)

print my_s
```

Μ.Ι.Σ. 2025-2026

ΟΙ ΕΙΔΙΚΟΙ ΧΑΡΑΚΤΗΡΕΣ ΔΙΑΦΥΓΗΣ

Εκτός από τους χαρακτήρες που είναι ορατοί, (πρέπει δηλαδή να εμφανίζονται), υπάρχουν και κάποιοι ειδικοί χαρακτήρες οι οποίοι δεν έχουν οπτική απεικόνιση με κάποιο ειδικό σύμβολο, αλλά όταν εμφανίζονται πρέπει να κάνουν μια ενέργεια.

Για παράδειγμα η αλλαγή της γραμμής στην οθόνη γίνεται με τον ειδικό χαρακτήρα νέας γραμμής. Για να μπορεί η γλώσσα να χειριστεί (να καταλάβει) έναν τέτοιο ειδικό χαρακτήρα και την ειδικότητα που αυτός αντιπροσωπεύει πρέπει να καθορίσουμε κάτι ειδικό για την αποτύπωση του,

το **η** δηλαδή δεν θα είναι για την γλώσσα ο χαρακτήρας **η**, αλλά θα αντιπροσωπεύει την αλλαγή γραμμής, με την προϋπόθεση όμως να γραφεί πριν από το **η** και ο χαρακτήρας ****

το **\η** είναι δηλαδή ένας χαρακτήρας διαφυγής, η Python όπως όλες οι γλώσσες διαθέτει χαρακτήρες διαφυγής (**escape characters**)

****	ανάποδη κάθετος (backslash)
\'	μονά εισαγωγικά
\"	διπλά εισαγωγικά
\n	νέα γραμμή
\t	tab

Μ.Ι.Σ. 2025-2026